Before we look at the `FileWatcher` implementation, here are the various exceptions that this application considers.

Listing 5.3 Exceptions in the log-processing application

```
@SerialVersionUID(1L)
class ClosedWatchServiceException(msg: String)      ❶
    extends Exception(msg)
    with Serializable

@SerialVersionUID(1L)
class ParseException(msg: String, val file: File)   ❷
    extends Exception(msg)
    with Serializable

@SerialVersionUID(1L)
class DbBrokenConnectionException(msg: String)      ❸
    extends Exception(msg)
    with Serializable

@SerialVersionUID(1L)
class UnexpectedColumnsException(msg: String)       ❹
    extends Exception(msg)
    with Serializable
```

❶ FileWatcherCapabilities exception

❷ LogProcessor’s parser exception

❸ DbWriter’s connector exceptions

❹ LogProcessor’s specific parser exception

These exceptions must be serializable because they must be sent over the wire. It is useful to note that all actors must also be serializable, but you don’t have to take care of it. Akka does it for you.

Listing 5.4 shows the file watcher with the proposed design that starts monitoring the directory at initialization—using `FileWatchingAbilities`. It creates and monitors a log processor that simplifies monitoring and provides the ability to